A Clean-Room Homemade Evaluation Stack: Direct SFNNv16 Comparison, Ablation, and Provenance

Manus AI
Unchessed UCI Engine Research
September 2026

***Abstract*—This paper documents a clean-room, independently authored evaluation stack for Unchessed. Stockfish 19 and SFNNv16 are used only as public architectural references and as an external black-box score target. No Stockfish source, network weight, feature row, implementation constant, or copied code is included in the project. The homemade stack combines eleven original bitboard-derived signal families, a conservative held-out feature selection policy, an independently fitted score shrinkage, and an original rational WDL projection. On 120 legal real-game positions, completed Stockfish 19 depth-10 UCI scores were compared with the homemade 'evalbar homemade' output. The selected stack achieved 178.925 cp mean absolute error, 103 cp median absolute error, and 1204 cp maximum error. The result is an engineering comparison, not a claim of SFNNv16 strength parity or superiority; that claim requires an independently trained network and statistically significant engine matches.**

I. Clean-Room Boundary

The design process used public Stockfish documentation only to understand general ideas: incremental state, sparse bitboard updates, quantization-friendly arithmetic, and training-aware calibration [1] [2]. The direct comparison used the official Stockfish 19 executable as a black-box UCI oracle and did not inspect its runtime memory, source implementation, network tensor, feature indices, or weights. The official network repository describes downloadable networks and their licensing [3], but none of those weights are copied into Unchessed.

All homemade code lives in 'unchessed-core/src/eval_bar.rs', the experiment CLI, and the reproducible tools under 'tools/'. The public boundary is enforced by provenance labels, a comparison script that exchanges only UCI text, and documentation that marks the result as a proxy. The runtime source label is 'homemade-stack-v3-selected'; official or loaded NNUE paths retain separate labels.

II. Original Breakthrough Stack

The homemade stack is a modular bitboard program rather than a reimplementation of SFNNv16. Its features are computed from current board occupancy and attack tables already exposed by the Unchessed engine. The broad research stack contains eleven signals, while the runtime uses a held-out selected subset.

TABLE I
Original Unchessed breakthrough families

Family	Homemade construction
Center-pressure lattice	Four-square central occupancy field
Pawn-geometry analyzer	Doubled, isolated, and connected-file structure
King-shelter ring	Own pawn cover in the king attack ring
Activity-space field	Extended-center occupation
Coordination mesh	Own pawn/knight attack overlap
Passed-pawn race	Adjacent-file enemy-ahead test
Outpost detector	Central knight squares outside enemy pawn attacks
Bishop-pair topology	Two-bishop same-side indicator
Rook-file pressure	Rooks on files without pawns
Tempo polarity	Side-to-move sign for experimental residual only
Phase-aware gain	Occupancy-derived phase multiplier

The runtime residual is bounded to $[-180, 180]$ cp. The selected production subset contains pawn geometry, king shelter, passed pawns, bishop-pair topology, and rook-file pressure. The other original signals remain in the implementation and ablation harness but are not allowed to add noise until a larger independent training set justifies them.

III. Independent Calibration

The score calibration belongs to Unchessed. It was fit from a held-out split of black-box UCI observations, using a linear shrinkage and bias after the homemade residual:

$$s_{home} = 0.680292396\, s_{raw} + 24.603188915. \tag{1}$$

These coefficients are not Stockfish coefficients and were not derived from Stockfish source or weights. The homemade WDL projection is also original: it uses a bounded rational curve whose scale depends on the homemade phase signal, followed by per-mille normalization. It is not the Stockfish WDL polynomial.

IV. Direct Comparison Protocol

The corpus contains 120 legal positions extracted from real annotated PGN games by 'tools/extract_pgn_positions.py'. It contains no engine scores or network data. For each FEN, the harness starts official Stockfish 19, requests depth 10, waits for 'bestmove',

TABLE II
DIRECT SCORE COMPARISON ON 120 LEGAL REAL-GAME POSITIONS

Metric	Selected homemade stack
Reference engine	Stockfish 19 / SFNNv16-era binary
Reference depth	10
Positions	120
Mean absolute error	178.925 cp
Median absolute error	103 cp
Maximum absolute error	1204 cp

TABLE III
ABLATION SUMMARY BEFORE FINAL RUNTIME RERUN

Variant	MAE cp
Base	180.442
Pawn geometry	177.633
King shelter	179.875
Bishop/rook	180.408
Full stack, calibrated	182.083
Selected stack, calibrated	177.183

records the final UCI score, then starts the Unchessed reviewer and records ‘evalbar homemade‘. Scores are normalized to White's perspective. This is a black-box numerical comparison; it is not a playing-strength match.

A prior full-stack calibration run on the same 120 positions measured 184.792 cp MAE. The selected runtime stack measured 178.925 cp MAE, an absolute reduction of 5.867 cp in this fixed corpus. The 30-position depth-8 check measured 231.200 cp MAE after the selected runtime change. These numbers are reproducible artifacts, not universal strength estimates.

V. Ablation and Performance

The ablation report evaluates feature families against the completed direct report. Before the final selection, pawn geometry was the most useful single family. A selected combination of pawn geometry, king shelter, passed pawns, bishop-pair topology, and rook-file pressure produced the best modeled generalization among the tested combinations. Full-stack inclusion was rejected because more signals increased variance on the available corpus.

The WDL tensor path remains separately optimized: the prior real-corpus benchmark measured 1,643,798 ns for the fast tensor path versus 4,974,389 ns for the exact logistic reference, a 3.03x projection speedup with zero per-mille WDL delta and identical checksums. This measures WDL conversion, not whole-engine nodes per second. Homemade feature extraction is used only by the on-demand eval-bar command and does not execute at every search node.

VI. Correctness and Provenance Tests

The eval-bar tests cover WDL conservation, symmetry, monotonicity, bounded smoothing, bitboard snapshot identity, incremental-state reuse, homemade source labeling, bounded residuals, and homemade WDL monotonicity. The complete repository suite must remain green before commit. The UCI smoke command is:

```
position startpos
evalbar homemade
```

The output includes ‘source=homemade-stack-v3-selected‘ and a read-only link containing the bitboard hash, occupancy, material index, Elo estimate, confidence, and detector reason.

VII. What the Evidence Does and Does Not Show

The homemade stack is original and now directly compared to an official Stockfish 19/SFNNv16-era black-box. It does not yet rival or surpass SFNNv16 in playing strength. A score MAE on 120 positions cannot establish Elo, tactical reliability, or search strength. A defensible superiority claim requires an independently trained network or evaluator, thousands of unseen positions, calibration metrics such as MAE and Brier score, and statistically significant engine matches under fixed hardware and time controls. The present work provides the clean-room feature stack, the comparison harness, and the measurement discipline needed for that next phase.

VIII. Conclusion

The project now contains a large set of homemade evaluator ideas, but it does not equate quantity with quality. Ablation is used to reject noisy components, calibration is learned only from black-box observations, and the runtime remains conservative. SFNNv16 informs the architectural questions; every implementation artifact, feature formula, score projection, and runtime policy in this update comes from Unchessed.

References

[1] Stockfish Team, “Stockfish 19,” Sep. 5, 2026. [Online]. Available: https://stockfishchess.org/blog/2026/stockfish-19/

[2] official-stockfish, “NNUE,” Stockfish Documentation. [Online]. Available: https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html

[3] official-stockfish, “Evaluation networks for Stockfish,” GitHub. [Online]. Available: https://github.com/official-stockfish/networks

[4] official-stockfish, “UCI Protocol and Stockfish Commands,” Stockfish Documentation. [Online]. Available: https://official-stockfish.github.io/docs/stockfish-wiki/UCI-Protocol-and-Stockfish-Commands.html